Implementation of Memory-Mapped Files in xv6-riscv

CS23B029 - Srikarthikkanithi

October 26, 2025

1 Implemented Functionality and Kernel Additions

This section details the structures and system calls implemented to introduce memory-mapped file support.

1.1 VMA (Virtual Memory Area) Management

The VMA structure is the foundation for tracking mapped regions within a process.

- **VMA Structure Definition (`proc.h`):**
  - Implemented a fixed-size array of VMA entries (maximum `NVMA` $= 16$ per process), as dynamic kernel memory allocation is avoided.
  - Each VMA stores essential metadata: `start`, `len`, access `prot`ection flags, `flags` (shared/private), associated `struct file` pointer (`f`), and file `off`set.
- **Constants (`fcntl.h`):** Defined protection flags (`PROT_READ`, `PROT_WRITE`, `PROT_EXEC`) and mapping flags (`MAP_SHARED`, `MAP_PRIVATE`).

1.2 System Call: `sys_mmap()`

Implemented the logic to establish a memory mapping, including file reference handling.

- **Setup and Validation:** Reads arguments, checks for page alignment, non-zero length, and valid flags.
- **VMA Allocation:** Finds a free VMA slot and allocates a suitable virtual address space region for the mapping.
- **File Reference Count:** Increments the reference count for the associated `struct file` to prevent premature deallocation upon file closure (similar to `filedup`).
- **Shared/Writable Check:** Verifies file permissions, returning -1 if attempting to set up a writable mapping on a read-only file.

1.3 System Call: `sys_munmap()`

Implemented the functionality to remove a memory mapping and synchronize changes.

- **VMA Identification and Unmapping:** Identifies the VMA associated with the target address range and utilizes `uvmunmap` to remove the designated pages from the process page table.

- **Synchronization (`MAP_SHARED`): Crucially, writes back modified pages** to the associated file before unmapping for `MAP_SHARED` mappings. This logic consults `filewrite` for implementation guidance and conceptually uses the PTE dirty bit (`D`) to identify modified pages.
- **File Cleanup:** Reduces the reference count for the associated `struct file` when the last page of a mapping is eliminated.

1.4 Kernel Integration: Page Faults, Fork, and Exit

Modified existing kernel components to correctly handle VMA regions and manage their lifecycle.

- **Page Fault Handler Modification:**
  - **Lazy Loading:** Checks the faulting address (`stval`) against the VMA table, verifies access permissions, and kills the process if protection is violated.
  - **Page Resolution:** Allocates a new page (for private/anonymous mappings) or retrieves the shared file page.
  - **File I/O:** Loads 4096 bytes from the corresponding file into the allocated page using `readi` with the correct offset (ensuring proper inode locking/unlocking).
  - **PTE Mapping:** Maps the page into the user address space with permissions configured by the VMA's `prot` flags.
- `kfork()` **Handling:** Copies the parent's VMA table to the child process. It duplicates file references and ensures page copying for private mappings while relying on lazy mapping for shared regions.
- `kexit()` **Handling:** Modifies `exit` to iterate over the VMA table, effectively invoking `munmap` for all VMAs to close file references and clear entries, preventing resource leaks.

1.5 Shared Mmap (`MAP_SHARED`) Support

Advanced features required for sharing memory between processes.

- **Shared Page Registry:** Implemented a registry with locking and reference counting to track globally shared physical pages.
- **Fault Handler Update:** Modified the page fault handler to lazily allocate shared physical pages, read file content only once, and correctly map shared pages on faults within `MAP_SHARED` VMAs.
- **Reference Count Management:** Developed logic to manage shared mapping reference counts across `fork()`, `munmap()`, and `exit()` to prevent premature page deallocation.

2 Development Difficulties and Solutions

This section outlines the technical challenges encountered during implementation and their corresponding resolutions.

- **Header and Type Issues:**
  - **Problem:** Missing header includes in files like `proc.c` and `sysfile.c`, and a type error due to not casting `buf` in `filewrite`.

  - **Solution:** Added necessary header includes and explicitly cast `buf` to `uint64` to match the kernel write signature.

- **Page Fault Panic:**
  - **Problem:** An initial implementation caused a "Panic: kerneltrap" error due to misinterpreting a condition in `usertrap` and using an incorrect handler.
  - **Solution:** Reverted the separate handler and correctly modified `vmfault()` to ensure proper fault resolution.

- **Read-Only File Safety Check:**
  - **Problem:** Initial implementations of `sys_mmap` and `sys_munmap` failed to check if a process was attempting to establish or utilize a writable mapping on a read-only file.
  - **Solution:** Added the necessary check in `sys_mmap` to return -1 and adjusted the logic in `sys_munmap` to correctly handle read-only file permissions.